Development Plan — Offline Bangla Dubbing Video Player

Oct 29, 2026 · @Md. Nazmul Hossain

Version 1 is built in 6 two-week sprints, turning the spike prototype into a packaged PyQt app for Windows, macOS and Linux. This plan follows BRD v1.1, Gap Analysis v1.1, Spike v1.1 and the Rework Plan.

Entry criteria

Development starts only when all of these are ticked:

- [ ] Spike go / no-go result is **Go** (E1 passed; no experiment hit its rework condition)
- [ ] Spike results recorded and any BRD changes approved by the product owner
- [ ] UI licence chosen: PySide6 (LGPL) or PyQt6 commercial
- [ ] Minimum and recommended hardware confirmed
- [ ] First Bangladeshi glossary drafted from spike experiment E9
- [ ] Team assigned: at least 2 developers, 1 QA, 1 product owner
- [ ] Code repository, issue tracker, CI and one GPU build machine set up

Architecture

The app has three layers. The UI never calls a model directly: it sends jobs to the pipeline, which runs in a separate worker process so playback never freezes.

Layer	Module	Responsibility	Requirements
UI (main process)	`player`	Video playback with mpv, audio-track and subtitle switching	BR-01, BR-11, BR-13
UI	`dub_panel`	Start dubbing, progress, mode choice (pre-process / live)	BR-08, BR-09, NFR-07

Layer	Module	Responsibility	Requirements
UI	`speaker_panel`	Speaker list, voice per speaker, rename / merge / split speakers	BR-05, BR-10, BR-17
UI	`line_editor`	Edit a Bangla line and regenerate its voice	BR-15
UI	`settings`, `model_manager`	Model download with resume and checksums, hardware detection, UI language	BR-07, BR-16, NFR-02, NFR-04
Pipeline (worker process)	`separate`	Demucs voice / background split; reports quality for the fallback	BR-06
Pipeline	`asr`	faster-whisper transcription and translate-to-English pivot	BR-02
Pipeline	`diarize`	pyannote speaker labels, limited to the configured maximum	BR-05
Pipeline	`translate`	IndicTrans2 to Bangla, then Bangladeshi glossary substitution	BR-03, BR-04a
Pipeline	`tts`	Indic Parler-TTS voice per speaker; swappable model for phase 2	BR-03, BR-05, BR-04b
Pipeline	`timing`, `mixer`	Fit lines to slots; mix with background or lowered original audio	BR-06, NFR-05
Pipeline	`scheduler`	Live mode: rolling buffer that stays ahead of playback	BR-09
Storage	`cache`, `export`	Results keyed by file hash; export WAV / SRT / MKV	BR-12, BR-14

UI and worker talk through a job queue with progress messages. Each pipeline stage writes its result to the cache, so a crash or cancel resumes from the last finished stage.

Tech stack and project structure

Area	Choice
Language	Python 3.11
UI	PyQt6 (or PySide6 for a closed-source LGPL app)
Playback	python-mpv with libmpv
AI	PyTorch, faster-whisper, pyannote.audio, Demucs, IndicTrans2, Indic Parler-TTS
Audio / video	ffmpeg
Worker process	`multiprocessing` with a job queue
Cache	SQLite index plus files per video
Tests	pytest, pytest-qt
Packaging	PyInstaller (Nuitka if startup is slow); per-OS installers
CI	GitHub Actions on Windows, macOS and Ubuntu runners

```
bangla-dub/
├── app/
|   ├── main.py            # entry point
|   ├── ui/                # player, dub_panel, speaker_panel, line_editor,
settings
|   ├── pipeline/          # separate, asr, diarize, translate, tts, timing,
mixer, scheduler
|   ├── core/              # jobs, cache, export, model_manager, hardware
|   └── i18n/              # Bangla and English UI strings
├── tests/
|   ├── unit/
|   ├── integration/       # short clips through the full pipeline
|   └── fixtures/          # test clips and expected outputs
├── packaging/             # PyInstaller specs, installer scripts per OS
├── docs/
└── pyproject.toml
```

The spike's `dub_pipeline.py` is split into the `pipeline/` modules; `spike_player.py` becomes `ui/player.py`.

Module interfaces

Every pipeline stage follows the same contract, so stages can be tested alone, cached, resumed and swapped.

```
class Stage(Protocol):
    name: str                    # "asr", "diarize", ...
    version: str                 # bump when output format or model changes

    def run(self, job: Job, inputs: StageInputs, progress: Callable[[float,
str], None]) -> StageResult:
        """Read input files, write output files into job.work_dir, return a
result object.
        Must not import UI code or open network connections."""
```

Stage	Input	Output
`extract`	Video file	`original.wav` (44.1 kHz stereo)
`separate`	`original.wav`	`vocals.wav`, `background.wav`, separation quality score

Stage	Input	Output
`asr`	`vocals.wav`	Lines with start, end, source text, English text, detected language
`diarize`	`vocals.wav`, lines	Speaker label per line
`translate`	Lines (English)	Bangla text per line, glossary terms applied
`tts`	Lines, speaker → voice map	One WAV per line
`timing`	Lines, line WAVs	Stretch factor and start time per line
`mixer`	Line WAVs, background	`dubbed.wav`
`export`	Video, `dubbed.wav`, lines	`.bn.srt`, `.bn.mkv`

A change in the UI (new voice, merged speaker, edited line) marks only the affected lines as dirty; `tts`, `timing` and `mixer` re-run for those lines only (BR-17, BR-15).

Data model and cache

Each video gets a folder under the cache root, named by a SHA-256 hash of its first 64 MB plus file size, so renaming a file keeps its dub (BR-14).

```
cache/<hash>/
├── job.json         # video path, settings, stage versions, status per stage
├── lines.json       # one entry per line (below)
├── speakers.json    # speaker id → display name, voice
├── stages/          # outputs per stage
└── tts/00001.wav …  # one clip per line
```

```
{
  "id": 42, "start": 61.20, "end": 63.85,
  "source_text": "…", "english": "Where were you last night?",
  "bangla": "কাল রাতে তুমি কোথায় ছিলে?",
  "speaker": "SPK_2", "voice": "Aditi",
  "edited": false, "stretch": 1.12, "status": "ok"
}
```

A SQLite file at the cache root indexes all videos (hash, title, last opened, size) for the recent-files list and cache cleanup.

Configuration

All model names and tuning values live in `config/default.toml`; users override them in Settings, which writes `user.toml`.

Key	Default	Purpose
`asr.model`	`large-v3` (`turbo` on minimum hardware)	Whisper size
`diarize.max_speakers`	6 (4 guaranteed)	BR-05
`translate.model`	`ai4bharat/indictrans2-en-indic-dist-200M`	Translator
`translate.glossary`	`glossary/bd_bangla.tsv`	BR-04a word list
`tts.model`	`ai4bharat/indic-parler-tts`	Swapped in phase 2 (BR-04b)
`tts.voices`	Arjun, Aditi, Tapan, Rashmi, Arnav, Riya	Voice order for new speakers
`timing.max_speedup`	1.35	Sync vs naturalness
`separate.min_quality`	0.6	Below this, use the audio-lowering fallback
`live.buffer_seconds`	45	BR-09 start delay
`paths.cache`	OS cache folder	Where dubs are stored

Error handling and fallbacks

The app never stops playback because of a dubbing problem. Each fallback below is the one written into BRD v1.1.

Situation	Behaviour	Requirement
One line fails in TTS or translation	That line plays original audio; marked in the line editor	NFR-06

Situation	Behaviour	Requirement
Separation quality below threshold	Mix the dub over the original audio lowered to 25%	BR-06
Bangla line too long after 1.35× speed-up	Keep it; flag it in the line editor for shortening	NFR-05, BR-15
More speakers than the maximum	Merge the smallest speakers into the nearest voice; tell the user	BR-05, BR-17
Hardware below recommended	Hide live mode; offer pre-process with smaller models	BR-09, NFR-02
Out of GPU memory	Retry the stage on CPU or with a smaller model; show a notice	NFR-06
Model file missing or corrupt	Model manager re-downloads when online; otherwise explains what is missing	BR-07, NFR-04
Worker process crashes	UI restarts it and resumes from the last cached stage	—
Unsupported source language	Dub via English pivot and label it "best-effort"	BR-02

Logging and privacy

- Logs stay on the device in the app's log folder, rotated at 10 MB, kept for 7 days.
- Logs record stage timings, errors and hardware; never transcripts, translations or file contents (NFR-03).
- No telemetry. A "Copy diagnostics" button lets users share a log voluntarily.
- The only network code is in `model_manager`; CI fails if any other module opens a connection.

Sprint plan

Six two-week sprints (12 weeks). Start date is set once the entry criteria are met. Each sprint ends with a working build on all three platforms.

Sprint	Goal	Scope	Requirements	Demo at sprint end
1	Foundation	Repo, CI on 3 OSes, app shell, player, worker process and job queue, config, model manager	BR-01, BR-07, NFR-04	Play a video; download models with progress
2	Pre-process pipeline	Port spike stages into modules; cache; glossary in translate; progress in UI	BR-02, BR-03, BR-04a, BR-08, BR-14	Dub a single-speaker English or Hindi video from the UI
3	Multi-speaker	Diarization, voice assignment, speaker panel with rename / merge / split	BR-05, BR-10, BR-17	Dub a 4-speaker clip; merge two speakers and re-dub
4	Quality and fallbacks	Timing fit, separation fallback, line editor, audio switch, subtitles, export	BR-06, BR-11, BR-12, BR-13, BR-15, NFR-05, NFR-06	Edit a line, regenerate, export MKV; fallback on a music-heavy clip
5	Live mode	Rolling-buffer scheduler, hardware check, auto-disable below recommended hardware	BR-09, NFR-02, NFR-10	Start a film within 60 s on the recommended GPU
6	Release readiness	Bangla UI, installers, performance, bug fixing, user docs	BR-16, NFR-01, NFR-07, NFR-09	Installers pass smoke tests; build handed to QA

BR-04b (Bangladeshi accent) is phase 2: a parallel track fine-tunes Indic Parler-TTS on SUBAK.KO. The new model drops in by changing `tts.model` in the config, with no code or UI change. If live mode (BR-09) misses its target in sprint 5, it moves to version 2 without delaying release.

Change capacity: each sprint keeps 20% of developer time free for approved change requests (CRs). After UAT, a 1-week **fix sprint** handles class A, B and E changes, and any class C change that blocks UAT approval once its re-spike has passed.

Engineering practices

Branching

- `main` is always releasable; `develop` collects finished work.
- One branch per issue: `feature/BR-05-speaker-panel`, `fix/123-sync-drift`, `cr/CR-001-llm-post-edit` for change requests.
- Every change goes through a pull request with 1 reviewer and green CI; CR pull requests link the approved CR.

Code standards

- Formatting and linting with ruff; type hints checked with mypy on `core/` and `pipeline/`.
- Each pipeline stage has one function: input files in, output files and a result object out. No UI imports in `pipeline/`.
- No network calls outside `model_manager`; a test fails the build if one appears.
- Model names, sizes and settings live in one config file, not in code.

Testing during development

Test type	What it covers	When it runs
Unit	Timing fit, mixer, cache, SRT writer, voice assignment	Every pull request
Integration	30-second clips through the full pipeline with small models	Every pull request (CPU)
GPU integration	5-minute test clips with full models; records timings	Nightly on a GPU machine
UI	Player controls, panels (pytest-qt)	Every pull request
Offline	Full run with network blocked	Nightly

Developer setup

1. Install Python 3.11, ffmpeg and libmpv (Ubuntu: `sudo apt install ffmpeg libmpv-dev`; macOS: `brew install ffmpeg mpv`; Windows: ffmpeg plus `libmpv-2.dll` on PATH).
2. Clone the repo and create a virtual environment: `python -m venv .venv`.
3. Install PyTorch for your hardware, then `pip install -e ".[dev]"`.
4. Accept the gated model pages on Hugging Face (pyannote segmentation 3.0 and diarization 3.1, Indic Parler-TTS, IndicTrans2), then set `HF_TOKEN`.
5. Run `python -m app.tools.fetch_models` once to download all models.
6. Start the app with `python -m app`; run tests with `pytest`.
7. For fast iteration, set `asr.model = "small"` in `user.toml` and use the 30-second clips in `tests/fixtures`.

CI, packaging and release

Stage	Runs on	Does
Lint and type check	Every pull request	ruff, mypy
Unit, UI, CPU integration tests	Every pull request, 3 OSes	pytest, pytest-qt, 30-second clips with small models
No-network check	Every pull request	Tests run with network blocked; any connection outside `model_manager` fails
GPU integration	Nightly, GPU machine	5-minute fixture clips with full models; timings stored to track NFR-10
Build installers	Tag on `main`	PyInstaller builds per OS

Platform	Installer	Notes
Windows	`.exe` installer (Inno Setup)	Bundles ffmpeg and libmpv; CUDA PyTorch build
macOS	Signed and notarised `.dmg`	Apple Silicon build; MPS PyTorch
Linux	AppImage, plus `.deb`	Bundles ffmpeg and libmpv

Models are not bundled in installers (NFR-04). Versions follow semantic versioning: `1.0.0` is the first release to UAT.

Roles and responsibilities

Role	Responsible for
Product owner	Priorities, approves sprint demos and BRD changes, decides on fallbacks
Lead developer	Architecture, pipeline modules, code reviews, release builds
Developer	UI modules, cache, export, packaging
QA	Test plan, nightly GPU results, bug triage with the product owner
ML engineer (phase 2)	SUBAK.KO cleaning and TTS fine-tuning for BR-04b
UAT testers	Native Bangladeshi Bangla speakers; score sprint demos from sprint 3

Development risks

Risk	Mitigation
Library version changes break model loading	Pin versions in `pyproject.toml`; nightly GPU test catches breakage
Installer size too large (PyTorch, CUDA)	CPU and GPU installer variants; models downloaded separately
macOS notarisation of bundled libraries	Start signing in sprint 1, not sprint 6
Diarization below target in real films	BR-17 tools ready in sprint 3; tune per genre
GPU machine is a bottleneck	Book one dedicated nightly machine before sprint 1

Handling change requests

Development works only on approved CRs from Feedback and Change Management; an unapproved request is logged as a CR, not coded.

CR class	Arrives from	What development does
A. Minor	Triage directly	Fix in the current sprint, inside the 20% change capacity
B. Requirement tweak	Updated BRD	Change config or acceptance logic; update tests
C. New technical approach	Passed re-spike (step 3)	Add the new component behind a config switch; keep the old one until QA passes
D. New feature	BRD and gap analysis (and re-spike if risky)	Plan into a sprint as new work
E. Fallback	Product owner decision	Switch the fallback on in config; update release notes
F. Deferred	—	Nothing now; stays in the backlog

Rules for CR work

- The CR ID appears in the branch name, pull request and release notes.
- Class C components are swapped through the config file (`translate.model`, `tts.model`), so rolling back is a one-line change.
- Every CR adds or updates at least one test case; the Test Plan is updated with the CR ID.
- A CR that grows beyond its approved effort by more than 50% goes back to the change board before more work is done.
- After a CR is merged, the lead developer marks it **In progress → Verified** only once QA passes its test cases.

Effect on the plan: a CR that moves a sprint's scope or the release date updates the sprint table above, and this document's change log, with the CR ID.

Definition of done and handoff to QA

A feature is done when:

- [ ] Code is merged to `develop` after review
- [ ] Unit and integration tests pass on Windows, macOS and Linux
- [ ] It meets the requirement's acceptance condition in the BRD
- [ ] It works with the network off
- [ ] User-facing text exists in Bangla and English
- [ ] The issue is updated with what changed and how to test it
- [ ] For CR work: the CR ID is linked and its documents (BRD, Test Plan) are updated

Handoff to step 5, Testing (QA), happens when:

- [ ] All Must requirements are done
- [ ] Installers for all three platforms are built from `main`
- [ ] Release notes list features, known issues and supported hardware
- [ ] Test clips and expected results are in `tests/fixtures`
- [ ] Open bugs: none critical, at most 5 major

Requirement traceability

QA uses this table to build the test plan in step 5.

Requirement	Priority	Module(s)	Sprint	Verified by
BR-01 Player	Must	`player`	1	UI tests
BR-02 Language detect	Must	`asr`	2	Integration: English, Hindi, Korean clips
BR-03 Speech replaced	Must	`asr`, `translate`, `tts`, `mixer`	2	Integration; spike E1
BR-04a Bangladeshi vocabulary	Must	`translate` (glossary)	2	Unit (glossary); reviewer score
BR-04b Bangladeshi accent	Should	`tts`	Phase 2	UAT score

Requirement	Priority	Module(s)	Sprint	Verified by
BR-05 Voice per speaker	Must	`diarize`, `tts`, `speaker_panel`	3	4-speaker fixture
BR-06 Keep background	Should	`separate`, `mixer`	4	Music fixture; fallback test
BR-07 Fully offline	Must	`model_manager`	1	No-network CI check
BR-08 Pre-process mode	Must	All pipeline	2	Integration
BR-09 Live mode	Should	`scheduler`	5	Nightly GPU test
BR-10 Choose voices	Should	`speaker_panel`	3	UI tests
BR-11 Switch audio	Should	`player`	4	UI tests
BR-12 Export	Should	`export`	4	Unit; ffprobe check
BR-13 Subtitles	Should	`export`, `player`	4	Unit (SRT)
BR-14 Cache	Could	`cache`	2	Unit; replay without reprocessing
BR-15 Edit lines	Should	`line_editor`	4	UI tests
BR-16 Bangla UI	Could	`i18n`	6	Manual review
BR-17 Fix speakers	Should	`speaker_panel`	3	UI and integration
NFR-05 Sync	—	`timing`	4	Nightly: lines within 0.5 s
NFR-10 Speed	—	All pipeline	5	Nightly GPU timings

Change log

Version	Date	Change
1.2	2026-10-29	Added change request handling, 20% change capacity, post-UAT fix sprint, `cr/` branches and CR item in the definition of done, following Feedback and Change Management
1.1	2026-10-29	Aligned with BRD v1.1: BR-04a/04b, BR-17, fallbacks; sprints updated
1.1	2026-10-29	Added module interfaces, data model, configuration, error handling, logging, developer setup, CI and packaging, roles, risks, traceability
1.0	2026-10-29	First version